Berri Bot Project Documentation

Resume based interview workflow and test record

Prepared for Manda Vijaya Sri
Execution date 18 October 2026
Project E:\Berry Bot

This record explains how Berri Bot was run locally, how three resumes were uploaded, and how the supplied resume was used in a five-question demonstration interview. It records the actual questions, submitted demonstration answers, evaluations, repeated test results and screenshots in the requested order.

The completed sessions both returned 88.0 percent and Strong Match. The answers were authored for this demonstration and were not spoken, supplied or confirmed by Manda Vijaya Sri. The result measures the local heuristic demonstration, and does not establish the candidate's ability or constitute a hiring decision.

1 Introduction to the Chatbot

Berri Bot is a resume based interview practice and assessment application. Candidates upload a PDF or DOCX resume, configure a role and interview language, answer questions, and receive scores and feedback. Recruiters can inspect reports and record reviewed recommendations.

Purpose and features

The application supports candidate authentication, resume extraction, interview configuration, an animated interviewer, typed responses, browser speech controls, question evaluations, history and PDF reports. Its interface lists English, Telugu, Hindi, Tamil and Kannada. This run tested English with typed answers; microphone recognition and other languages were not verified.

1 Introduction to the Chatbot continued

How the system works

The React and TypeScript frontend calls a FastAPI backend. SQLAlchemy stores accounts, resumes, sessions and question evaluations in SQLite. The parser uses pypdf for PDF text and python-docx for Word files. The interview service builds a question plan from the structured resume; each answer is evaluated and stored. Completing the interview averages the answered-question scores and generates a recommendation.

The health endpoint identified version 2.0.0 and Berry Bot Heuristic AI Local Demo Mode. No cloud model was used. In this mode the question generator fills fixed language templates using the first project, first programming language, first framework and first database. Scoring uses answer length, technical keyword matches and solution words. A function for dynamic follow-ups exists, but the inspected interview route advances through the stored plan and did not insert follow-up questions in this test.

Criterion	Weight
Technical correctness	30 percent
Relevance	20 percent
Depth of understanding	20 percent
Problem solving	20 percent
Clarity of explanation	10 percent

An answer score is the weighted sum of the five dimension scores. The final score is the arithmetic mean of answered questions. These are five criteria on a 0 to 100 scale, rather than a score out of five.

Observed limitations

The first PDF upload produced a generic Full-Stack Web Application project and an incorrect degree, institution and GPA. Its extracted email joined adjacent PDF text. Correcting projects, education, experience, summary and topics through the existing resume update API fixed the main question context. That API does not persist candidate email or phone updates; the malformed parsed email remains visible in screenshots. The original PDF is the authoritative resume.

The heuristic evaluator does not substantively check correctness or expected topic coverage. For example, the system-design answer did not explain idempotency-key storage or atomic deduplication but scored 90.5. The system-design question also described a REST API in React, illustrating mechanically substituted templates. Strong scores should therefore be read as software outputs requiring human review.

2 Resume Upload

Three PDF files were uploaded through the API and then through the browser interface. Resume one is the user supplied original. Resumes two and three are fictional profiles distributed with this project, converted to PDF for upload testing. They are not additional real candidates. The main interview uses only the corrected user profile.

Resume	Source	Use
Manda Vijaya Sri	Supplied PDF	Main interview and repeat test
Priya Sharma	Built-in fullstack sample	Additional PDF upload test
Rahul Varma	Built-in aiml sample	Additional PDF upload test

Upload and review findings

All three PDFs were accepted. Successful extraction did not guarantee faithful field parsing. For the main resume the uncorrected test sessions 7 and 8 generated generic project questions. The primary completed browser session 11 and repeat session 12 used corrected resume record 27. Earlier configured or exploratory records visible in History are not the final comparison.

The reviewed profile retains the four project names from the PDF: Quantum Traffic Optimization System, CarePulse Healthcare Management System, Drone Detection AI, and Fresh Cart MERN Web App. Education was corrected to B.Tech AI and DS at Seshadri Rao Gudlavalleru Engineering College, CGPA 9.01 till date, and Diploma CME at VKR, VNB and AGK College, CGPA 9.13. Internship organizations were corrected to SmartInternz, Indian Servers and SkillDzire.

Resume one original document

The next page reproduces the supplied resume. The two following profiles reproduce the project sample data in a readable record format. Original and generated upload PDFs are included in the accompanying project folder.

Resume one Manda Vijaya Sri

Manda Vijaya Sri

Gudivada, Andhra Pradesh 521301

6301749467 vijayasrimanda025@gmail.com linkedin.com/in/vijaya-sri-manda github.com/vijayasri-manda

Professional Summary

To build a rewarding career in a dynamic and technology-driven organization where I can apply my technical knowledge, enhance my skills, and contribute to the organization's success while achieving continuous professional growth.

Education

Course	Institution	University/Board	Year	CGPA
B.Tech (AI&DS)	Seshadri Rao Gudlavalleru Engineering College	JNTU Kakinada	Pursuing	9.01 (Till Date)
Diploma (CME)	VKR, VNB & AGK College	SBTET Andhra Pradesh	2021–2024	9.13

Experience

SmartInternz
Full Stack (MERN) Intern | *React.js, Node.js, Express.js, MongoDB, Git* May 2025 – Jul 2025
- Developed responsive full-stack web applications using the MERN stack with scalable frontend interfaces.
- Built RESTful APIs and integrated MongoDB databases to handle dynamic user data and system state.

Indian Servers
Cybersecurity Intern | *Nmap, Wireshark, Burp Suite, SQLmap* Apr 2025 – Jul 2025
- Conducted vulnerability assessments and penetration testing using Nmap, Wireshark, and Burp Suite.
- Identified web application security vulnerabilities and prepared comprehensive mitigation reports.

SkillDzire
Data Science Intern | *Python, Pandas, NumPy, Matplotlib* May 2025 – Jun 2025
- Performed exploratory data analysis on complex datasets using Pandas and NumPy to uncover key trends.
- Created insightful data visualizations using Matplotlib to support strategic data-driven decision-making.

Projects

Quantum Traffic Optimization System | *Python, Qiskit, Streamlit, Google Maps API* | Live Demo | GitHub Feb 2025
- Built a quantum-inspired traffic optimization engine using Qiskit algorithms and integrated Google Maps API for dynamic real-time route computation.
- Engineered an interactive Streamlit dashboard featuring visual route analytics, live traffic congestion tracking, and optimized path recommendations.

CarePulse – Healthcare Management System | *React.js, Spring Boot, MySQL* | Live Demo | GitHub Jan 2025
- Architected a scalable full-stack healthcare platform equipped with modular management flows for patients, doctors, and appointment scheduling.
- Implemented secure JWT authentication, robust Spring Boot REST APIs, and optimized relational database queries using MySQL.

Drone Detection AI | *YOLO, OpenCV, Computer Vision, Firebase* | Live Demo | GitHub Dec 2024 – Jan 2025
- Engineered a real-time Computer Vision pipeline leveraging YOLO object detection models to accurately identify unauthorized aerial drones in live video feeds.
- Architected serverless hosting on Firebase to achieve low-latency model inference, fast video stream processing, and live public access.

Fresh Cart (MERN Web App) | *React.js, Node.js, Express.js, MongoDB* | GitHub Nov 2024 – Dec 2024
- Designed a full-stack e-commerce platform using the MERN stack with dynamic product cataloging and interactive shopping cart management.
- Optimized MongoDB database queries and REST API routes to ensure fast response times for product searches, user orders, and inventory updates.

Certifications

Microsoft – Machine Learning Operations Engineer Associate (AI-300) Jun 2025
Microsoft – Azure AI Cloud Developer Associate (AI-200) May 2025
Oracle – Agentic AI Certified Foundations Associate Apr 2025
Microsoft – Azure AI Fundamentals (AI-900) Feb 2025
Google – Gemini Certified University Student Jan 2025
Salesforce – Certified Agentforce Specialist Nov 2024
MongoDB – Associate Developer Exam (Node.js) Oct 2024
Udemy – Deep Learning Foundations for AI and Gen AI Aug 2024

Technical Skills

Languages: Python, Java, C, JavaScript, HTML/CSS, SQL
Developer Tools: VS Code, Google Cloud Platform, AWS, Google Colab, Jupyter Notebook, Git, Firebase, Docker
Technologies and Frameworks: React.js, Node.js, Express.js, Spring Boot, MongoDB, PostgreSQL, Pandas, Matplotlib
Core Concepts: Machine Learning, Computer Vision, Cybersecurity, Data Analytics, Microservices, RESTful APIs

Resume two Priya Sharma

Fictional project sample profile

priya.sharma@example.com | +91 98451 23456

Full Stack Developer with 2+ years of experience designing scalable web applications. Proficient in React, TypeScript, Node.js, and PostgreSQL. Experienced in micro-frontends, REST/GraphQL APIs, and Docker containerization.

Skills

Languages: JavaScript, TypeScript, Python, SQL, HTML5, CSS3

Frameworks: React, Next.js, Node.js, Express.js, Tailwind CSS

Databases: PostgreSQL, MongoDB, Redis

Tools And Platforms: Git, Docker, AWS (S3, EC2), Jest, Postman, CI/CD

Other: REST API, GraphQL, Agile/Scrum, Microservices Architecture

Projects

HealthSync: Telemedicine Appointment Portal

Technologies: React, TypeScript, Node.js, PostgreSQL, Socket.io

Architected a real-time doctor-patient consultation platform serving 5,000+ monthly active users with end-to-end appointment scheduling.

- Implemented real-time queue management with WebSockets and Redis pub/sub

- Designed normalized PostgreSQL schemas with composite indexes reducing query latency by 45%

- Built role-based access control (RBAC) with JWT authentication and refresh token rotation

ShopSmart: High-Throughput E-Commerce Platform

Technologies: React, Redux Toolkit, FastAPI, Stripe API, Docker

Engineered an e-commerce platform with dynamic inventory tracking, server-side caching, and secure payment processing.

- Optimized React bundle size by 35% through dynamic code-splitting and memoization

- Containerized microservices using Docker Compose for staging and production parity

Experience and education

Nexus Technologies Pvt Ltd | Associate Software Engineer | Jul 2023 - Present

- Developed and maintained React customer portal components with 95% test coverage

- Collaborated with backend engineers to design idempotent RESTful APIs

- Participated in bi-weekly sprint planning and code reviews

Bachelor of Technology in Computer Science & Engineering | JNTU College of Engineering, Hyderabad | 2019 - 2023 | 8.7 / 10 CGPA

Certifications: AWS Certified Cloud Practitioner, Meta Frontend Developer Professional Certificate

Resume three Rahul Varma

Fictional project sample profile

rahul.varma@example.com | +91 99123 45678

AI/ML Engineer specializing in Computer Vision, Natural Language Processing, and production ML pipelines. Proficient in Python, PyTorch, Hugging Face, FastAPI, and MLflow.

Skills

Languages: Python, C++, SQL, R

Frameworks: PyTorch, TensorFlow, FastAPI, Scikit-Learn, Hugging Face Transformers

Databases: PostgreSQL, Milvus (Vector DB), Redis

Tools And Platforms: Docker, Kubernetes, MLflow, Git, Weights & Biases, AWS SageMaker

Other: Deep Learning, Transfer Learning, Model Quantization (ONNX), REST APIs

Projects

MediScan: Automated Chest X-Ray Diagnostic Classifier

Technologies: Python, PyTorch, TorchVision, FastAPI, Docker

Trained and deployed a DenseNet-121 model to detect pneumonia and cardiomegaly from 100k+ radiological images with 94.2% ROC-AUC.

- Mitigated severe class imbalance using focal loss and dynamic data augmentation

- Exported model to ONNX runtime, achieving 3x inference acceleration on CPU endpoints

- Packaged FastAPI service in a lightweight Docker container deployed on AWS ECS

LegalDoc-Summarizer: Multilingual Document Insight Engine

Technologies: Hugging Face, LangChain, Milvus, Python

Built a Retrieval-Augmented Generation (RAG) system for Indian legal contracts across English and Hindi.

- Segmented multi-page legal contracts into semantic chunks with overlapping token windows

- Integrated Milvus vector search with hybrid BM25 re-ranking for accurate clause retrieval

Experience and education

Cognitive AI Labs | Machine Learning Engineer Intern | Jan 2024 - Jun 2024

- Benchmarked transformer backbones for domain-specific sentiment analysis

- Automated data preprocessing pipelines using Python and Celery workers

Bachelor of Technology in Data Science & Artificial Intelligence | Vellore Institute of Technology (VIT) | 2020 - 2024 | 9.1 / 10 CGPA

Certifications: DeepLearning.AI Deep Learning Specialization, TensorFlow Developer Certificate

3 Procedure

1 Start the backend

Open PowerShell in E:\Berry Bot\backend and run .\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000. The no-reload command avoids the Windows named-pipe failure encountered with run.py.

2 Start the frontend

The normal developer command is npm run dev in frontend. In this environment Vite failed to bundle native Tailwind dependencies and spawn a helper. The existing frontend/dist build was served with documentation/serve.py, which proxies /api to port 8000. Run the bundled Python executable with documentation/serve.py from the repository root.

3 Verify the running service

Open http://127.0.0.1:5173 and inspect http://127.0.0.1:8000/api/health. Confirm the healthy status and local heuristic provider.

4 Sign in or register

Create a candidate account with the candidate name. This test used a dedicated documentation account, not the resume email. Demo Candidate is also available for general practice.

5 Upload all three resumes

Choose Resume Analysis, select the PDF and wait for the successful extraction message. Repeat for both sample PDFs. Upload the user resume last so it is active for the main interview.

6 Review the parsed profile

Compare the extracted projects, skills, education and experience with the original. In this run structured corrections were submitted through PUT /api/resumes/{id}, because the visible editor only changes the summary. The submitted correction is retained in corrected_profile.json.

3 Procedure continued

7 Configure the interview

Select Proceed to Interview Config. Use Full Stack Developer, Fresher, Mixed, English, 15 minutes and five questions. Confirm the selected resume is the corrected user record.

8 Enter the interview

Click Enter Live AI Interview and review Audio Setup and Privacy Consent. This demonstration used the typed mode and did not test the microphone.

9 Answer and evaluate

Choose Type Answer, enter a response and click Submit Answer. Read the immediate rubric and feedback. Capture the output, then choose Continue to Next Question. Repeat for all five questions.

10 Finish and inspect

On question five choose Complete and View Final Report. Review the score, recommendation, question transcript, strengths and knowledge gaps. Use the report download control for the application report if needed.

11 Repeat and compare

Create a new session with the same resume and settings. Submit exactly the same five answers. Compare question text, criterion scores, feedback and recommendation. This repeat was executed against the same live API, with its results visible in History.

12 Prepare the record

Retain exact questions and answers, raw JSON, screenshots and source prompt templates. Build this PDF and inspect every rendered page before sharing.

Verification outcome

The backend test suite completed with 9 passed and 36 warnings. Warnings include deprecated library APIs and pytest cache permission messages. The observed resume fidelity and heuristic scoring limitations remain despite those passing tests. Frontend development build startup was not successful; the existing compiled UI was used for browser testing.

4 Prompts Used

Build and documentation instruction provenance

The historical conversational prompts used to create the original project are not present in the supplied repository or this chat. They cannot be reconstructed as factual records. The available implementation prompts are reproduced below. They are source templates, not cloud requests executed during this local demo.

Documentation request used for this work: Prepare a document in the order Introduction to the Chatbot, Resume Upload, Procedure, Prompts Used, Interview Questions and Answers, Repeated Question Testing, Final Selection Decision, and Screenshots. First run the project and use Vijaya_Sri_Manda_Resume.pdf.

Operating configuration

```
{
"job_role": "Full Stack Developer",
"experience_level": "Fresher",
"interview_type": "Mixed",
"language": "en",
"duration_minutes": 15,
"total_questions": 5,
"resume_id": 27
}
```

The resume_id value identifies the corrected user upload. Operation requests were POST /api/interviews, POST /api/interviews/{id}/start, POST /api/interviews/{id}/answer with answer_text and input_type text, and POST /api/interviews/{id}/next-question. GET /api/interviews/{id}/report retrieved the recorded result. The application has no general free-form user prompt box; it asks questions and accepts answers.

Testing inputs

The exact five local question prompts, five submitted answers and returned feedback are printed in section 5. The repeated test resubmitted those strings without edits under the same settings. Section 6 records the repeated responses and score comparison. No new conversational build prompt was executed to alter the application code.

Implementation prompt 1

Gemini interview planning template

Source backend/app/services/ai_service.py. Braced values are runtime substitutions. This template was not sent to an external provider in the documented run.

```
You are Berri Bot, an expert AI Technical Interviewer at a premier technology enterprise.
Generate exactly {total_questions} structured interview questions tailored specifically for:
Job Role: {job_role}
Experience Level: {experience_level}
Interview Type: {interview_type}
Target Language: {settings.SUPPORTED_LANGUAGES.get(language, 'English')} (Language code: {language})

Candidate Resume Details:
{json.dumps(parsed_resume, indent=2) if parsed_resume else 'No resume provided; generate standard
industry questions.'}

Job Description:
{job_description or 'Standard industry benchmark'}

CRITICAL RULES:
1. Root the questions in the candidate's actual projects, frameworks, and listed tools. NEVER invent
projects or qualifications.
2. If language is 'te' (Telugu), 'hi' (Hindi), 'ta' (Tamil), or 'kn' (Kannada), write the
question_text naturally in that script, but keep technical terms (e.g. React, Spring Boot, REST API,
indexing) in standard technical form so it sounds professional and natural.
3. Provide question_text_en with the English translation.
4. Output valid JSON array with objects containing:
- "order_index": int (1 to {total_questions})
- "category": str (e.g. "Resume Project Deep Dive", "Technical Architecture", "System Design",
"Behavioral")
- "question_text": str (in target language)
- "question_text_en": str (in English)
- "audio_caption": str (same as question_text)
- "expected_topics": list of str
Return ONLY raw JSON, no markdown fences.
```

Implementation prompt 2

Gemini answer evaluation template

Source backend/app/services/ai_service.py. Braced values are runtime substitutions. This template was not sent to an external provider in the documented run.

```
You are Berri Bot's rigorous AI Answer Evaluation Engine.
Evaluate the candidate's answer based on the following rubric:
Question: {question_text}
Candidate's Answer: {candidate_answer}
Job Role: {job_role}
Level: {experience_level}
Category: {category}
Expected Key Topics: {json.dumps(expected_topics)}
Rubric Weights: {json.dumps(rubric_weights)}

CRITICAL EVALUATION GUIDELINES:
- Assess technical correctness, relevance, depth, problem solving, and clarity on a 0-100 scale.
- Calculate overall weighted score using the provided rubric weights.
- Evaluate the meaning and semantic correctness; DO NOT penalize accents, language dialect, or minor
grammatical imperfections.
- Distinguish between factual inaccuracies versus merely incomplete answers.
- Return ONLY a JSON object with:
"score_overall": float,
"score_technical": float,
"score_relevance": float,
"score_depth": float,
"score_problem_solving": float,
"score_clarity": float,
"feedback_justification": str,
"strengths": str,
"missing_concepts": str,
"ideal_answer_summary": str
No markdown fences.
```

5 Interview Questions and Answers

Primary browser session 11. All answers below are AI-authored demonstration inputs submitted on behalf of the workflow test. They must not be attributed to the candidate as actual interview testimony. The chatbot text below is transcribed from the live API record.

Question 1 Resume & Project Deep Dive

In your project 'Quantum Traffic Optimization System' built with Python, Qiskit, Streamlit, Google Maps API, can you explain the architectural structure, how data flows through the application, and the most challenging technical hurdle you personally resolved?

Submitted demonstration answer

For my Quantum Traffic Optimization System, the resume lists Python, Qiskit, Streamlit and Google Maps API. In this demonstration I would separate route data collection, optimization and dashboard presentation. I would validate missing traffic inputs, compare routes against a classical baseline and test response latency. The resume does not establish quantum advantage or measured performance, so those claims require supporting experiments.

Chatbot response

Candidate provided a thorough explanation. Demonstrated technical understanding in api, orm, latency. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: api, orm, latency, test.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Overall	Technical	Relevance	Depth	Problem solving	Clarity
85.9	95	90	86	68.3	85

Question 2 Core Programming & Concurrency

As a candidate for Full Stack Developer, could you explain how memory management, concurrency, and async execution work in Python? What pitfalls like deadlocks or memory leaks should an engineer watch out for?

Submitted demonstration answer

I would handle API concurrency with asynchronous I/O for network calls and a bounded worker pool for CPU intensive tasks. I would prevent shared state races using isolated request data and transaction boundaries. I would test simultaneous requests, measure latency and log failures before choosing a deployment scale. This is a proposed approach rather than a verified description of my implementation.

Chatbot response

Candidate provided a thorough explanation. Demonstrated technical understanding in state, async, transaction. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: state, async, transaction, api. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Overall	Technical	Relevance	Depth	Problem solving	Clarity
91.1	95	90	92	88.3	85

Question 3 Database & High Performance

When scaling applications with PostgreSQL, how do you design tables and indexes for high-throughput reads and writes? How would you identify and fix a slow-running query in production?

Submitted demonstration answer

I would design database indexes around frequent filters and joins, inspect the query plan and measure latency using representative data. To handle consistency I would use a transaction for related updates, validate inputs and test rollback paths. Indexes speed reads but consume storage and make writes more expensive, so I would retain only indexes justified by measurements.

Chatbot response

Candidate provided a thorough explanation. Demonstrated technical understanding in index, transaction, query. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: index, transaction, query, join. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Overall	Technical	Relevance	Depth	Problem solving	Clarity
90.5	95	88.5	92	87.1	85

Question 4 System Design & Idempotency

Design scenario: Imagine we need to build an idempotent REST API for payments or notifications in React that processes 5,000 requests per second. How do you guarantee zero duplicate operations and handle network timeouts?

Submitted demonstration answer

For a reliable API I would validate requests, enforce authentication and authorization, use parameterized database queries and return safe error messages. I would handle timeouts with bounded retry policies for idempotent operations. I would add tests, structured logs and metrics so failures can be reproduced and security issues investigated. This describes my proposed design for the scenario.

Chatbot response

Candidate provided a thorough explanation. Demonstrated technical understanding in api, idempotent, retry. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: api, idempotent, retry, test. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Overall	Technical	Relevance	Depth	Problem solving	Clarity
90.5	95	88.5	92	87.1	85

Question 5 Engineering Collaboration & Behavioral

Tell me about a situation where you had a disagreement with a team member or reviewer regarding a technical choice or pull request. How did you communicate, evaluate trade-offs, and reach a consensus?

Submitted demonstration answer

During a team disagreement I would first ask each member to explain the requirement and the evidence behind their preferred design. I would document the options, agree on a small test and compare the results against shared acceptance criteria. If the issue remained unresolved I would ask the project mentor to help decide. This is a hypothetical behavioral answer for the demonstration.

Chatbot response

Candidate provided a thorough explanation. Demonstrated technical understanding in test. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: test. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Deep architectural trade-offs and edge-case handling.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Overall	Technical	Relevance	Depth	Problem solving	Clarity
82.1	81.8	90	66.5	88.6	85

6 Repeated Question Testing

A second session 12 used the same corrected resume, role, level, language, interview type and five-question count. The exact questions from section 5 were generated again, and the identical answer text was submitted to each. Record IDs and timestamps differ between sessions; these metadata differences are expected.

Question	Same text	Run one	Run two	Feedback match
1	Yes	85.9	85.9	Yes
2	Yes	91.1	91.1	Yes
3	Yes	90.5	90.5	Yes
4	Yes	90.5	90.5	Yes
5	Yes	82.1	82.1	Yes

Both aggregate scores were 88.0 percent. Both recommendations were Strong Match. All five criterion scores, overall scores, strengths, missing concepts and ideal-answer summaries matched for each repeated question. This demonstrates repeatability of the local heuristic for identical inputs; it does not validate semantic accuracy or predict cloud model behavior.

Exact repeated chatbot responses

Repeated question 1 response

Candidate provided a thorough explanation. Demonstrated technical understanding in api, orm, latency. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: api, orm, latency, test.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Repeated question 2 response

Candidate provided a thorough explanation. Demonstrated technical understanding in state, async, transaction. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: state, async, transaction, api. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

6 Repeated Question Testing continued

Repeated question 3 response

Candidate provided a thorough explanation. Demonstrated technical understanding in index, transaction, query. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: index, transaction, query, join. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Repeated question 4 response

Candidate provided a thorough explanation. Demonstrated technical understanding in api, idempotent, retry. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: api, idempotent, retry, test. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Additional discussion of automated observability and telemetry.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

Repeated question 5 response

Candidate provided a thorough explanation. Demonstrated technical understanding in test. Explanation was clear and directly targeted to the role of Full Stack Developer.

Strengths: Demonstrated good awareness of core terms: test. Articulated a concrete engineering approach to solving the scenario.

Missing concepts: Deep architectural trade-offs and edge-case handling.

Ideal answer summary: An exemplary response outlines the architectural structure, evaluates specific trade-offs (e.g. latency vs consistency, memory vs CPU), and mentions production safeguards like automated retries, rate limiting, and observability.

7 Final Selection Decision

Recorded chatbot recommendation Strong Match

Overall score 88.0 percent in primary session 11 and repeat session 12.

Chatbot explanation verbatim

Candidate demonstrated comprehensive mastery for the Full Stack Developer (Fresher) profile, achieving an aggregate score of 88.0%. Responses showed high technical precision, sound architectural problem-solving, and clear articulation of design trade-offs.

Evidence used by the application

Dimension	Session score
Technical correctness	92.4
Relevance	89.4
Depth	85.7
Problem solving	83.9
Clarity	85

The final mean is (85.9 + 91.1 + 90.5 + 90.5 + 82.1) / 5 = 88.02, rounded to 88.0. The local recommendation thresholds are Strong Match at 80 or above, Potential Match from 65 to below 80, Further Assessment Recommended from 50 to below 65, and Does Not Yet Meet the Defined Criteria below 50.

Key Strengths: Q1 (Resume & Project Deep Dive): Scored 85.9%; Q2 (Core Programming & Concurrency): Scored 91.1%; Q3 (Database & High Performance): Scored 90.5%; Q4 (System Design & Idempotency): Scored 90.5%; Q5 (Engineering Collaboration & Behavioral): Scored 82.1%

Areas for Improvement: Continuous refinement in large-scale system patterns.

Interpretation and final authority

The software recommendation is recorded as requested. No actual candidate selection has been made. The responses were demonstration inputs and the local evaluator relies heavily on keywords and length. It awarded high scores even where an answer omitted the main requested concept. A recruiter should review the authentic resume, obtain the candidate's own answers, and verify technical claims before making any selection decision. No recruiter override was applied in these sessions.

8 Screenshots

Screenshots captured from the running application on 18 October 2026. Images show actual UI outputs; captions distinguish observed behavior from untested features.

Figure 1 Application landing screen

The running compiled frontend introduces the workflow and languages.

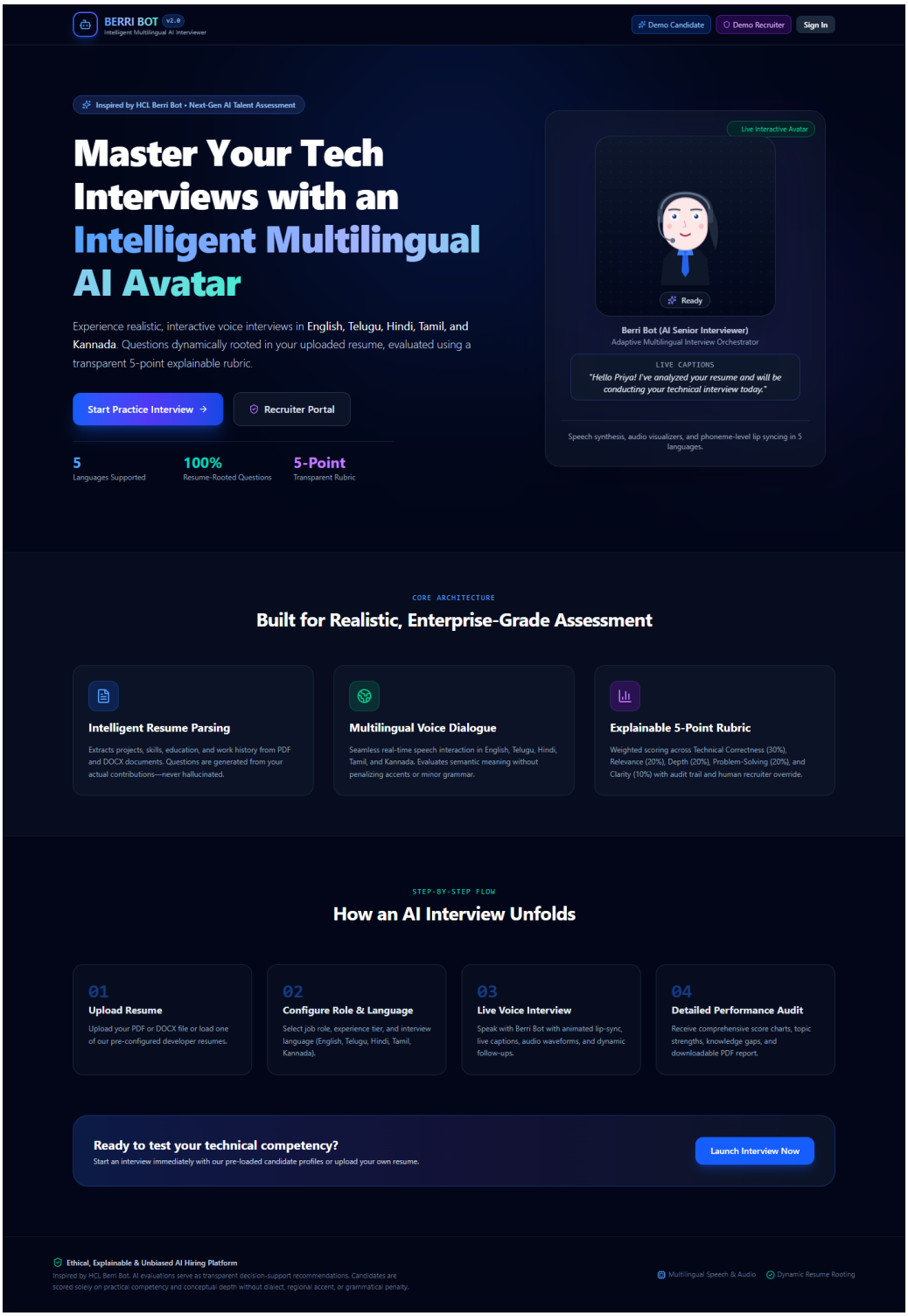


Figure 2 Additional resume one upload

The Priya Sharma sample PDF was accepted by the upload endpoint.

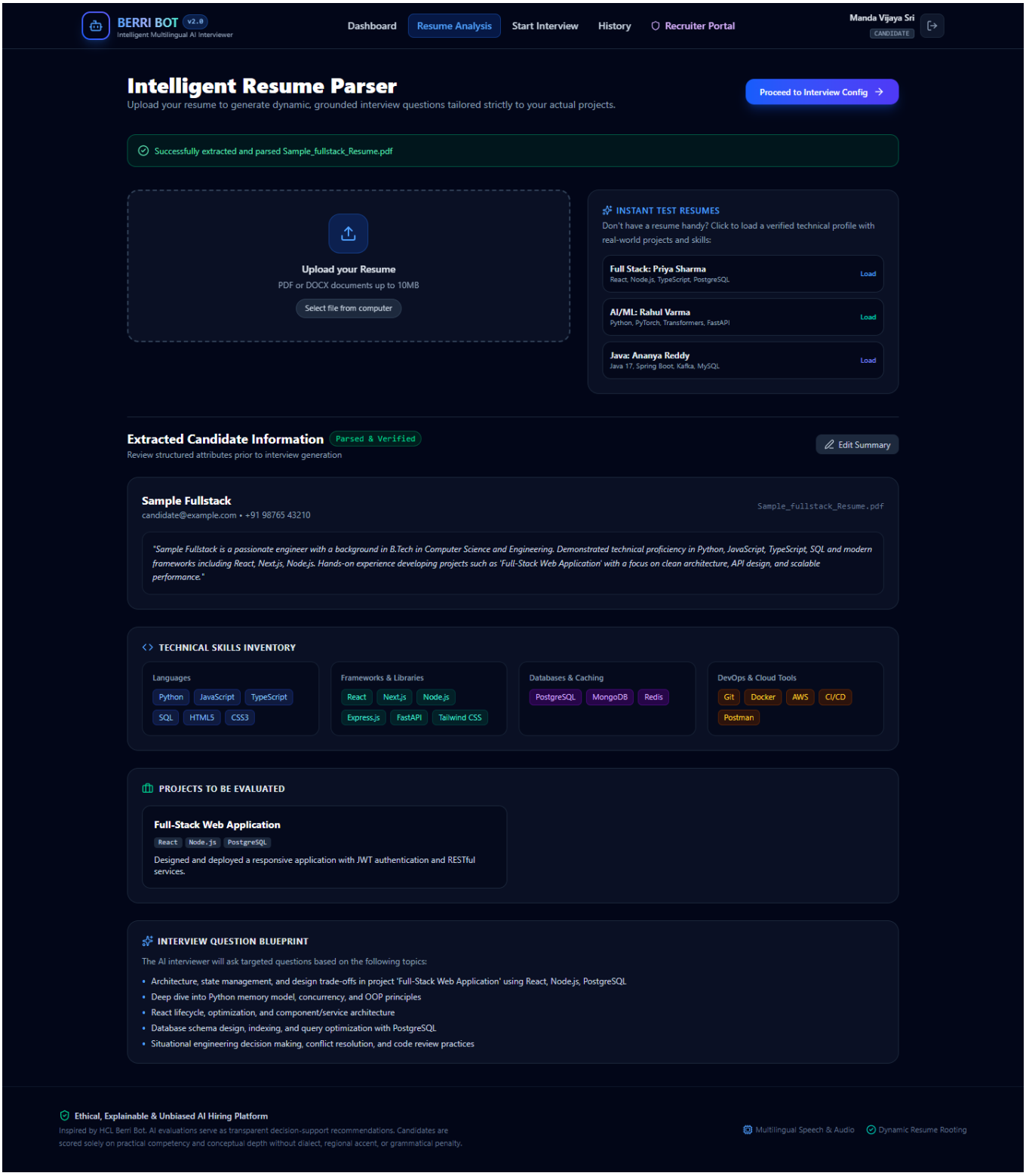


Figure 3 Additional resume two upload

The Rahul Varma sample PDF was accepted by the upload endpoint.

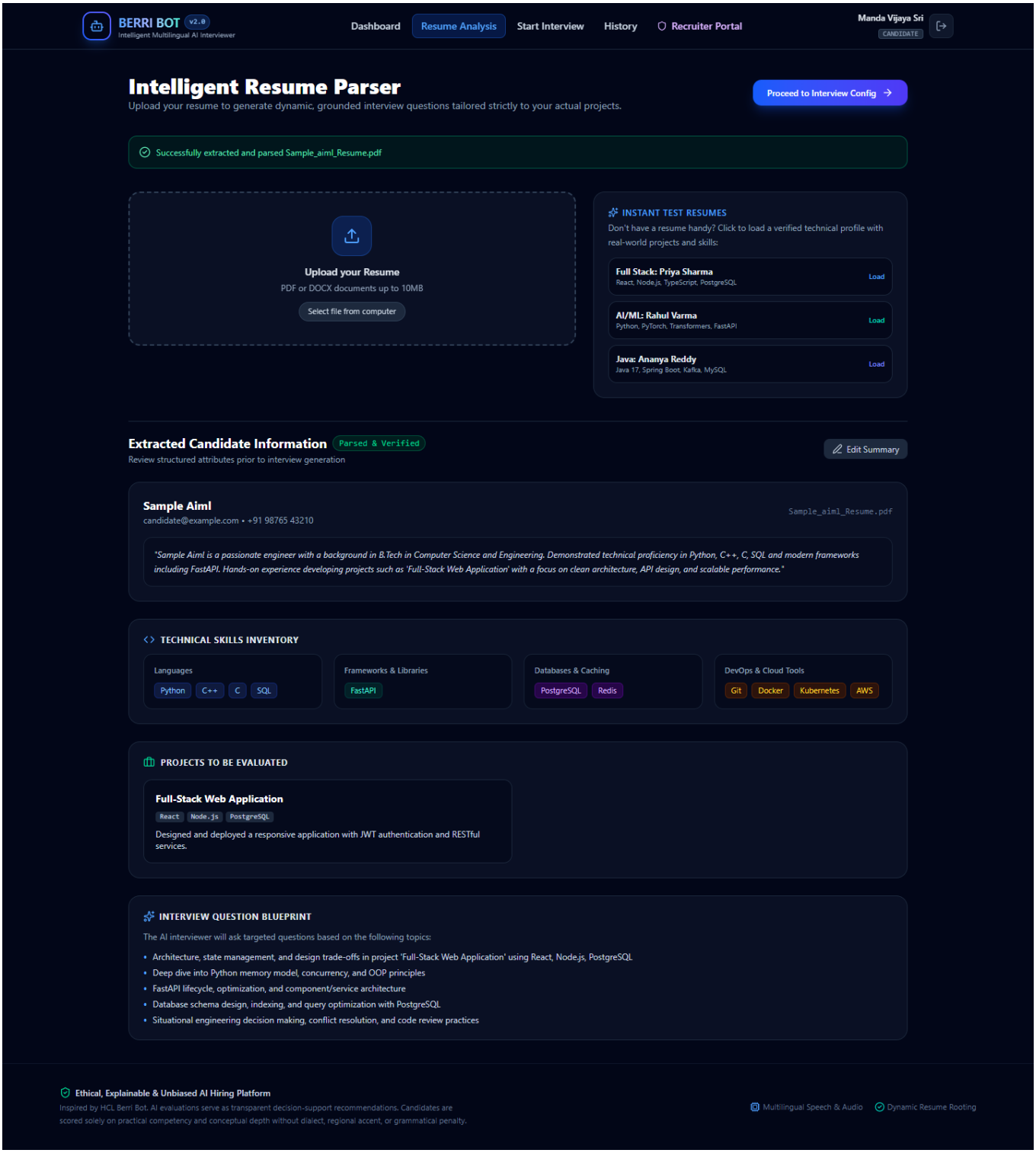


Figure 4 User resume upload before correction

Successful extraction message and generic project data illustrate the parser fidelity problem.

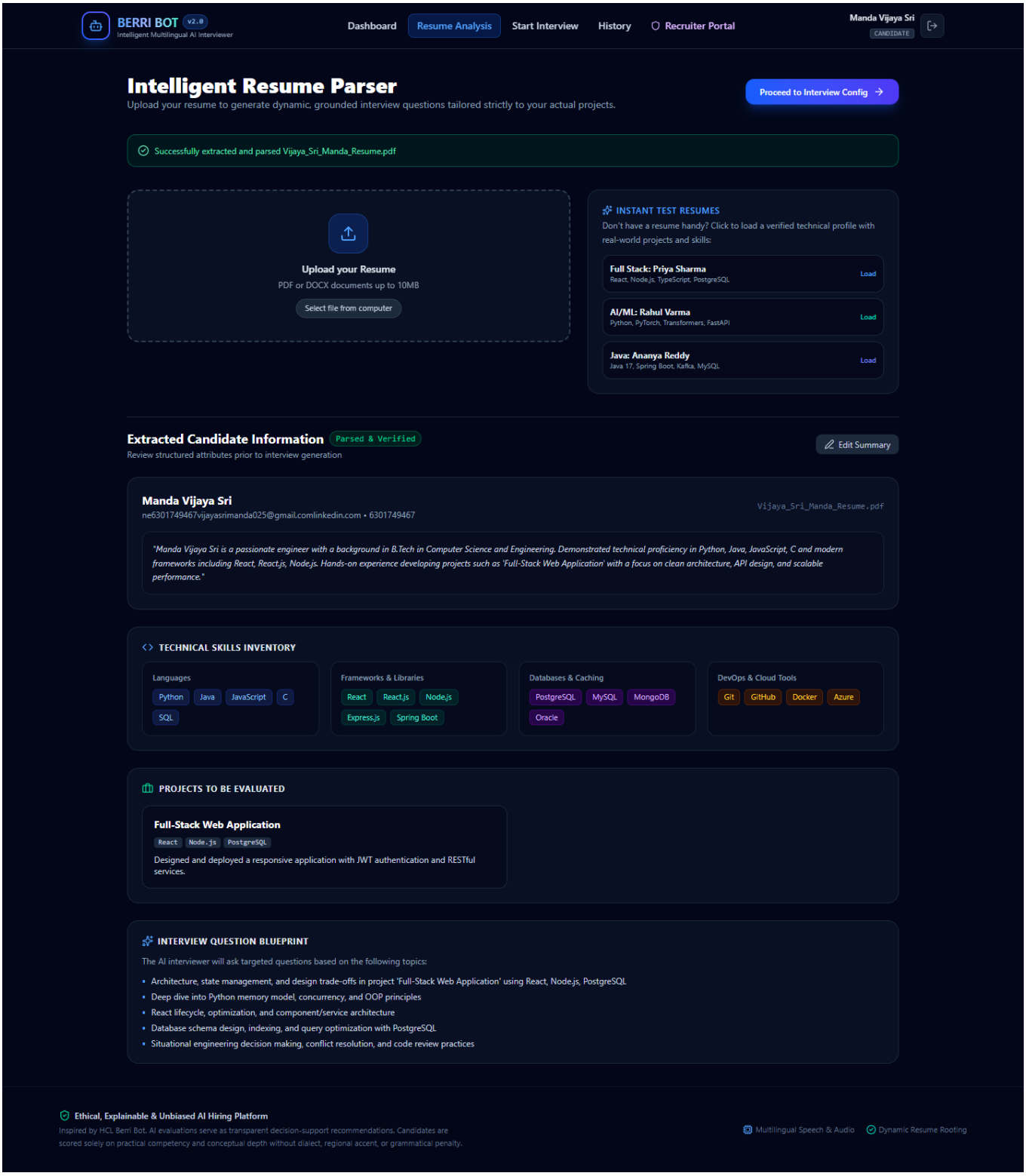


Figure 5 Reviewed user profile

Corrected project names are shown. The malformed parsed contact field remains an API limitation.

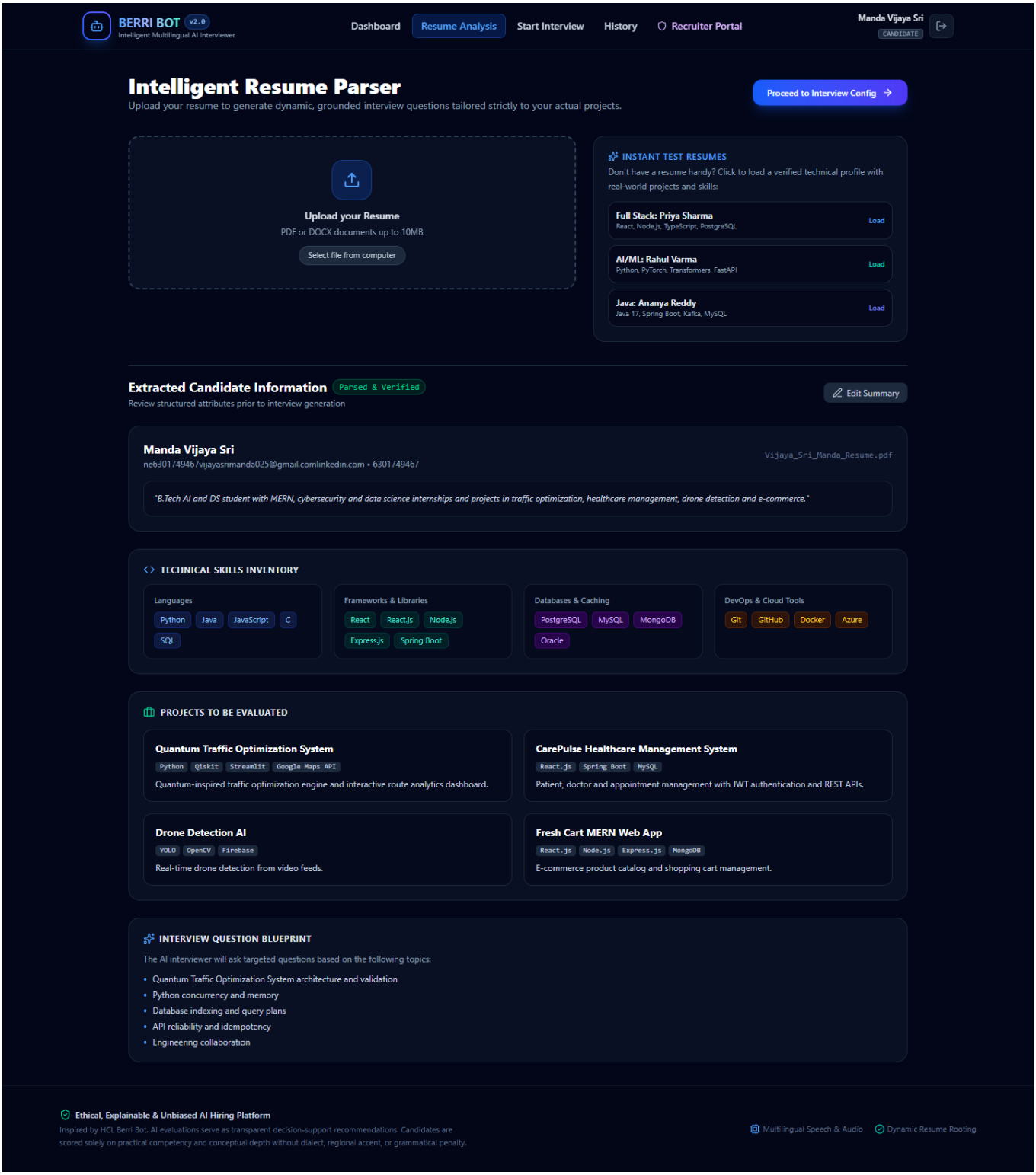


Figure 6 Interview configuration

Full Stack Developer, Fresher, Mixed, English, 15 minutes and five questions.

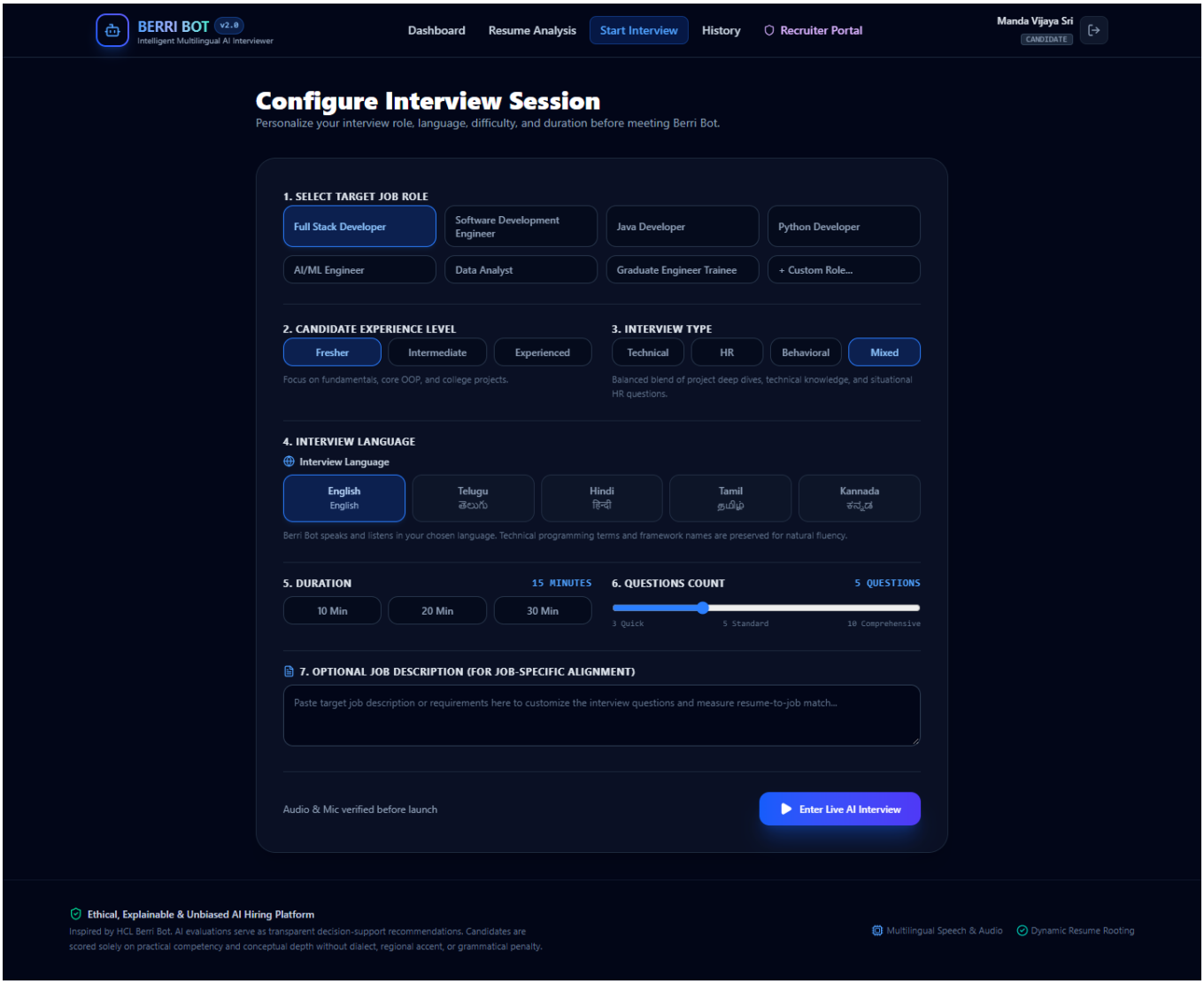


Figure 7 Audio setup and consent screen

The workflow proceeded with typed inputs. Microphone behavior was not tested.

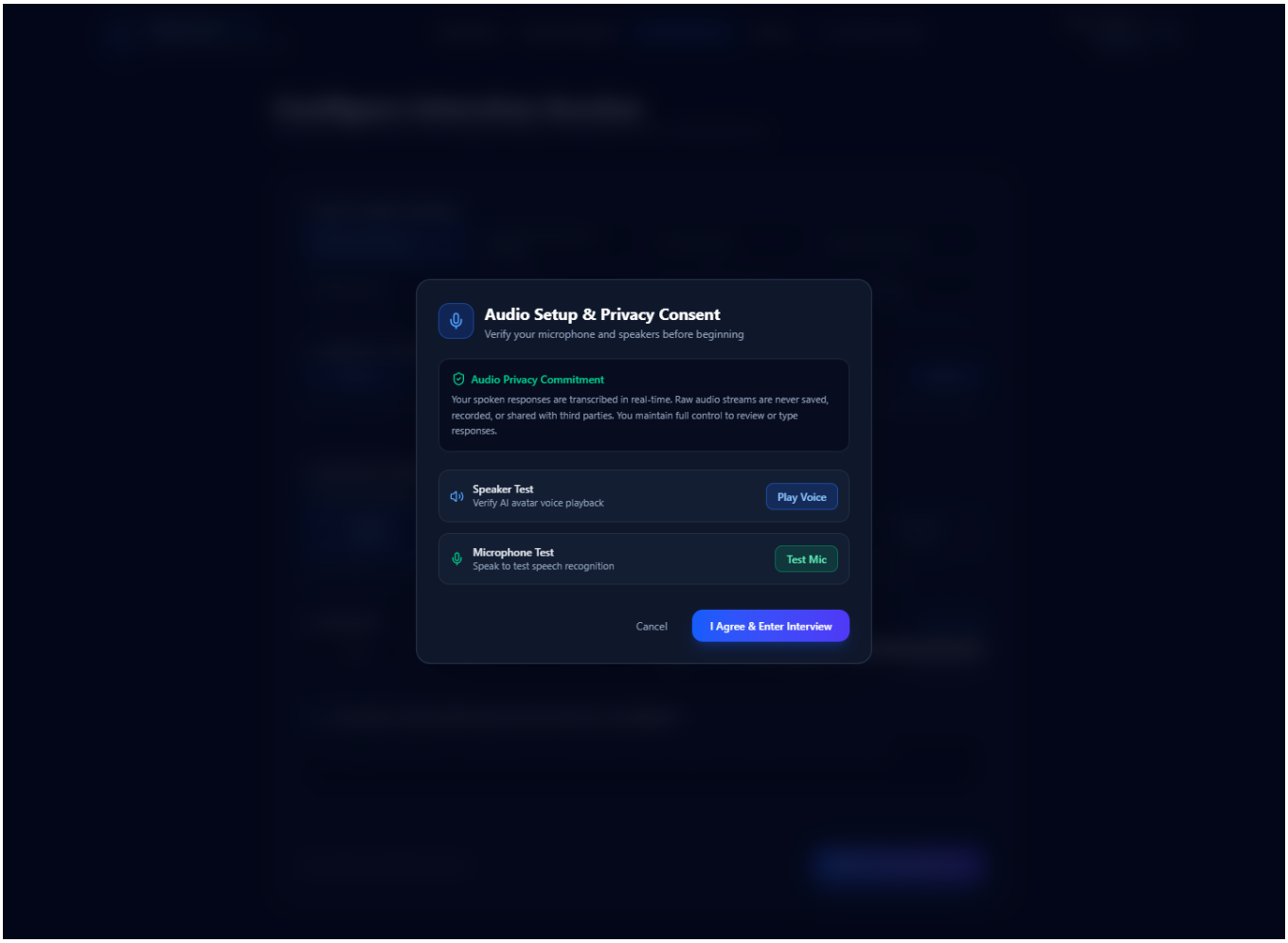


Figure 8 Live interview question

The first question uses Quantum Traffic Optimization System from the corrected resume.

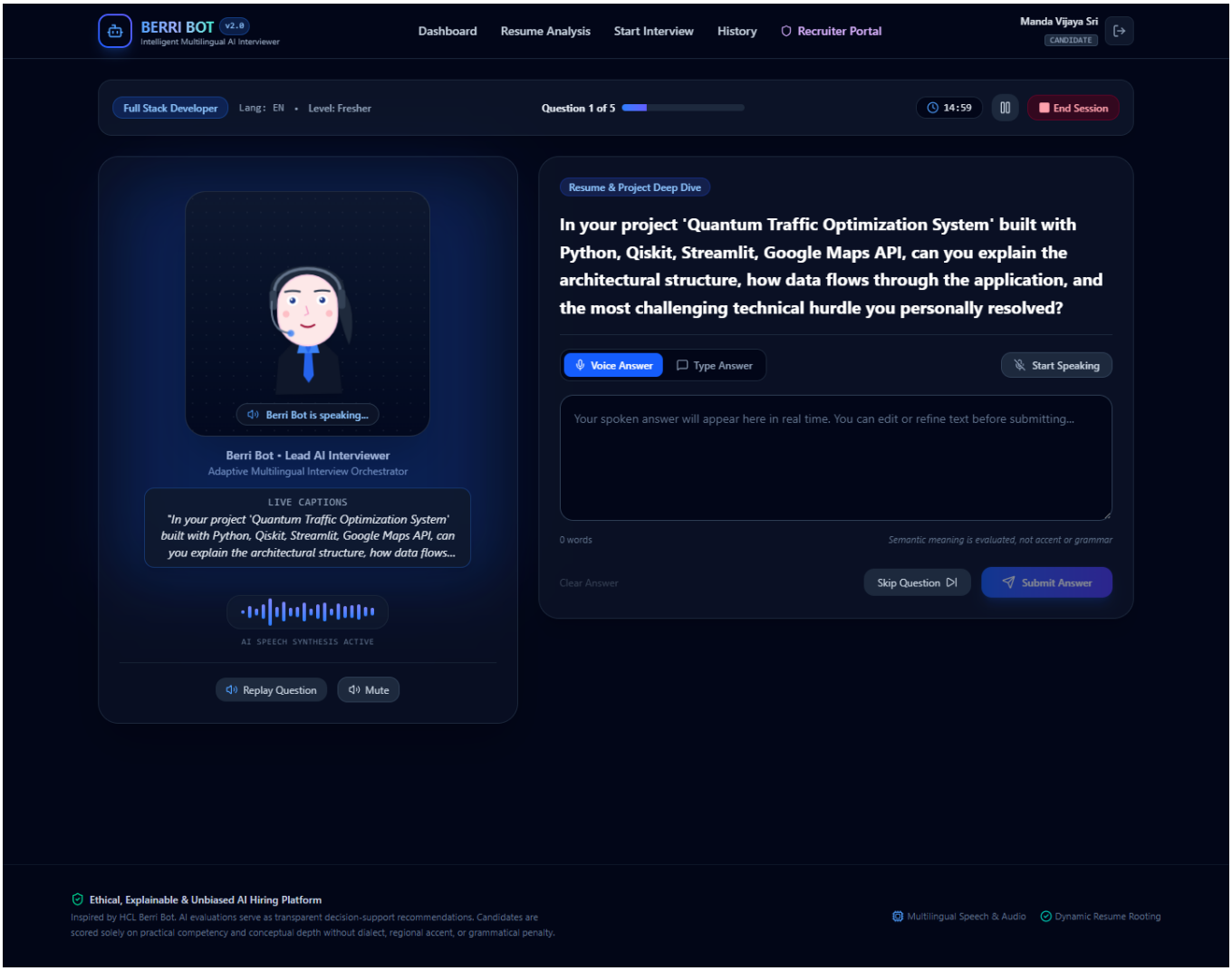


Figure 9 Question 1 evaluation

Real submitted demonstration answer and the returned rubric feedback.

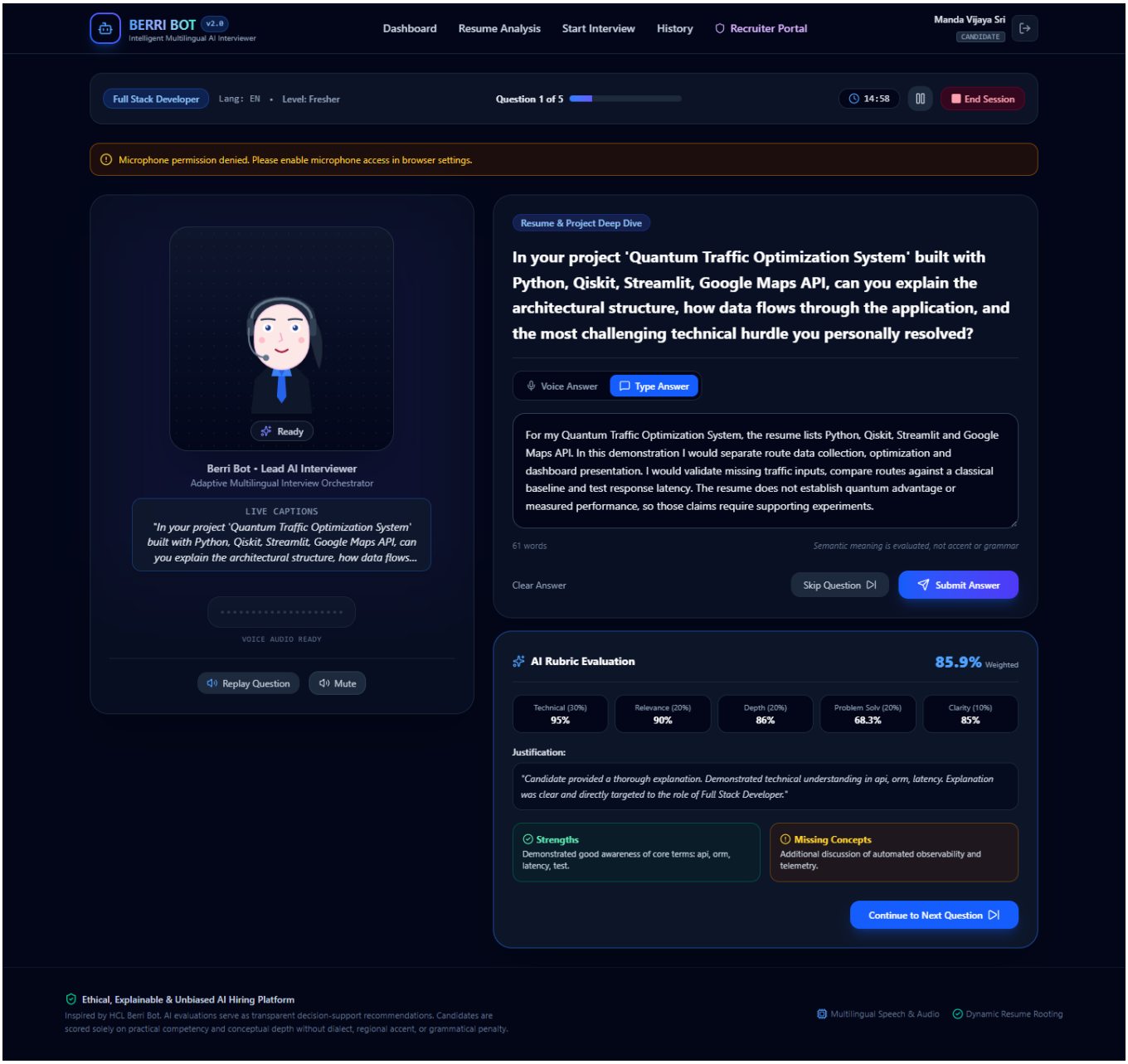


Figure 10 Question 2 evaluation

Real submitted demonstration answer and the returned rubric feedback.

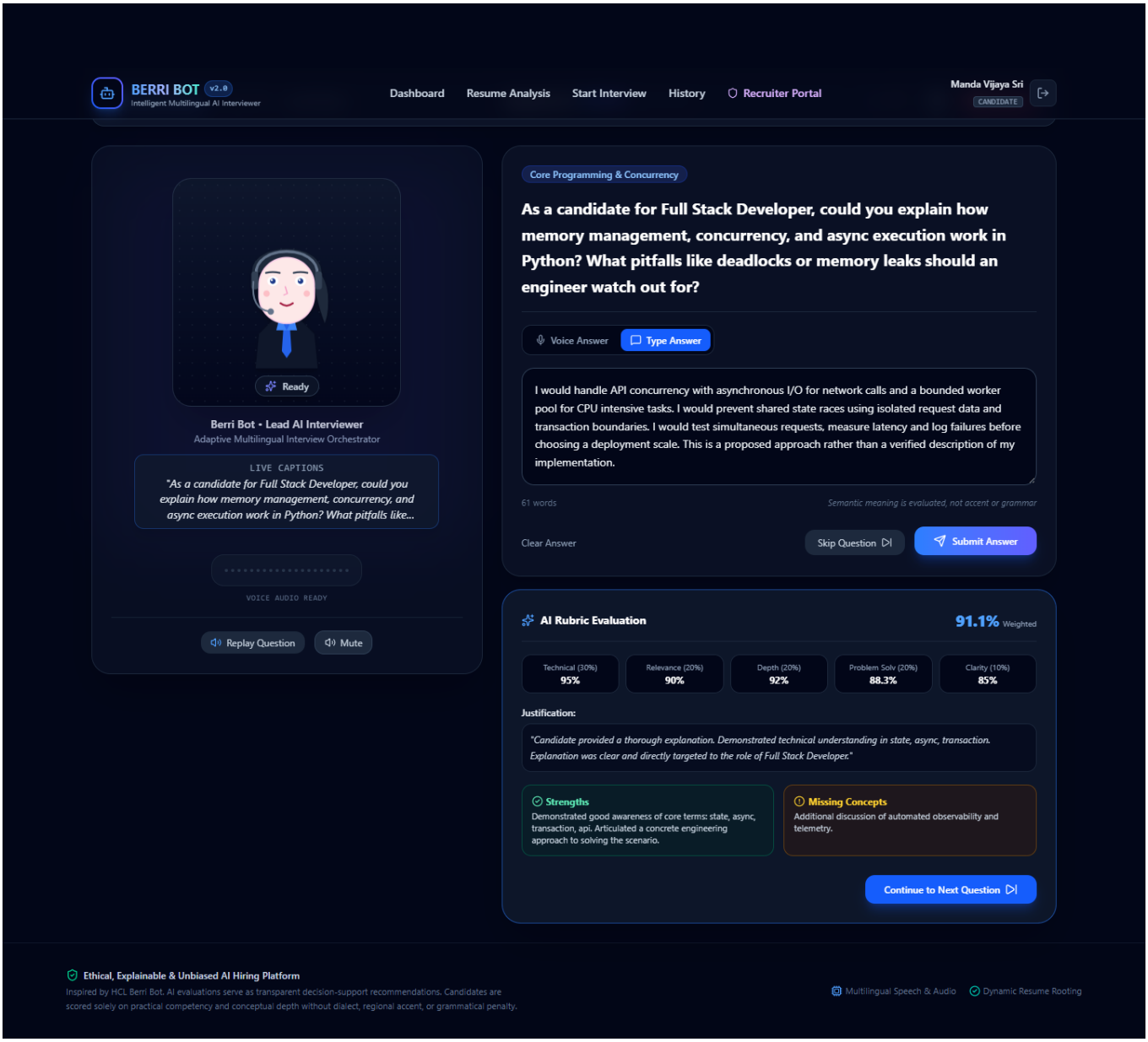


Figure 11 Question 3 evaluation

Real submitted demonstration answer and the returned rubric feedback.

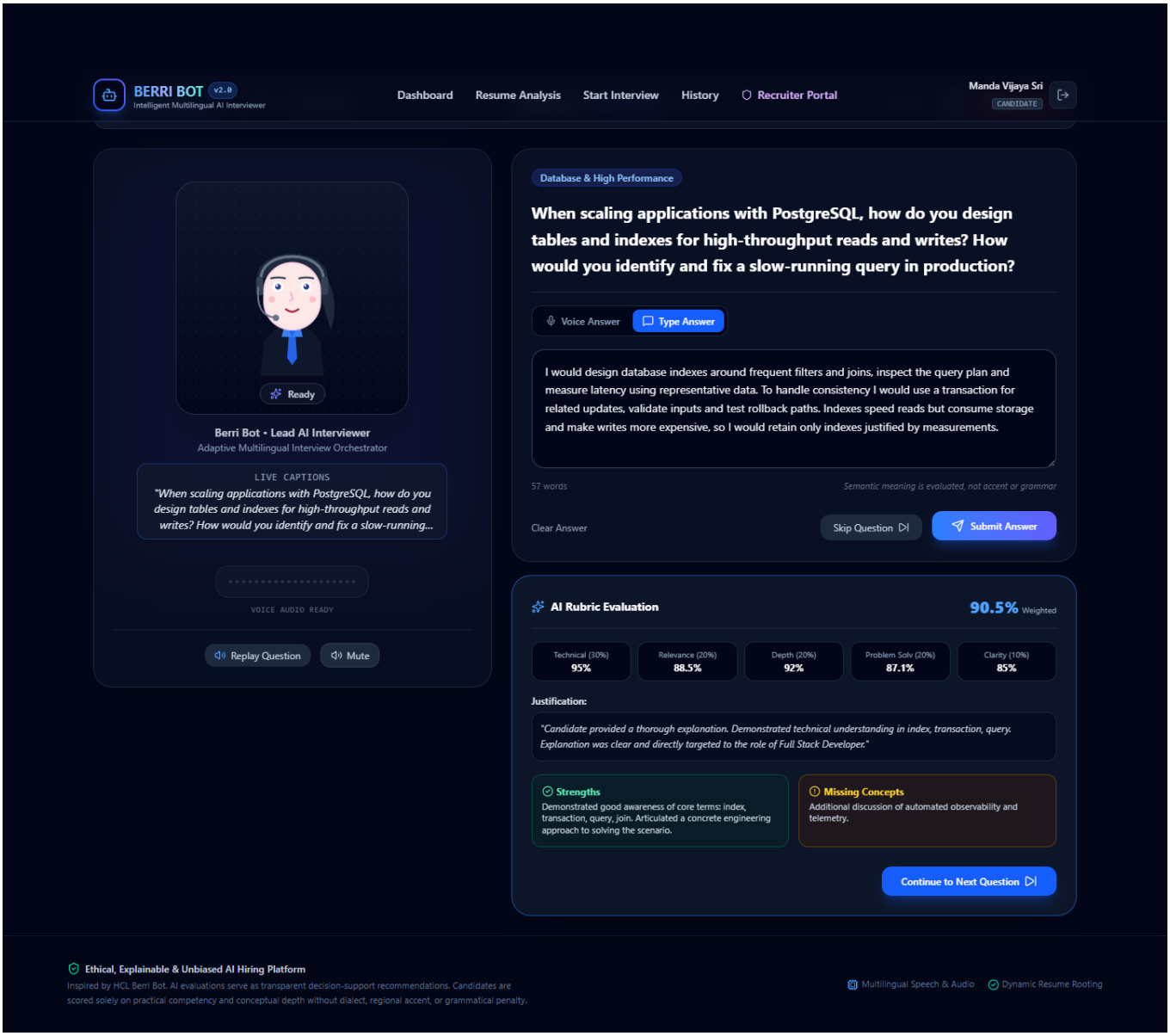


Figure 12 Question 4 evaluation

Real submitted demonstration answer and the returned rubric feedback.

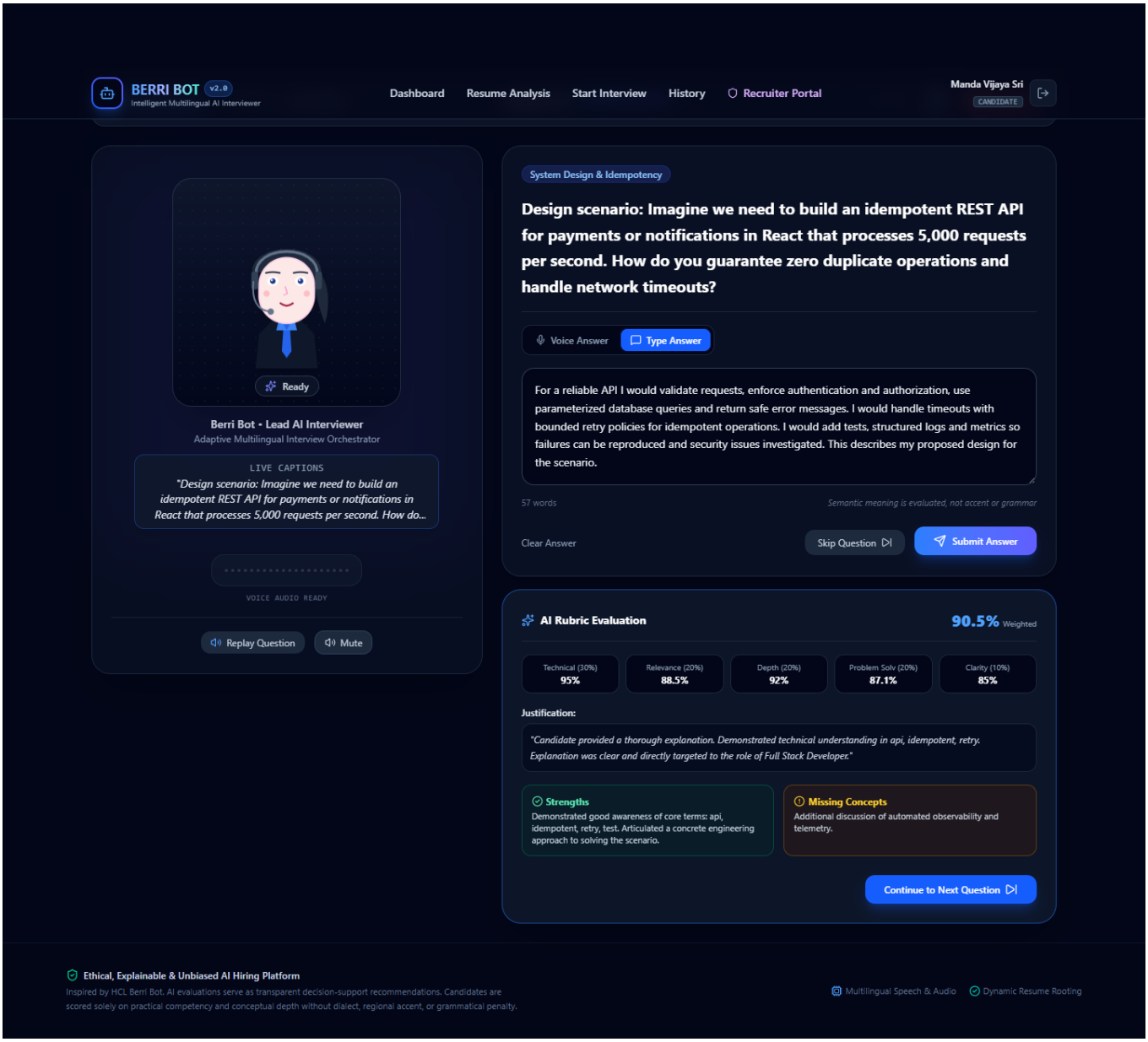


Figure 13 Question 5 evaluation

Real submitted demonstration answer and the returned rubric feedback.

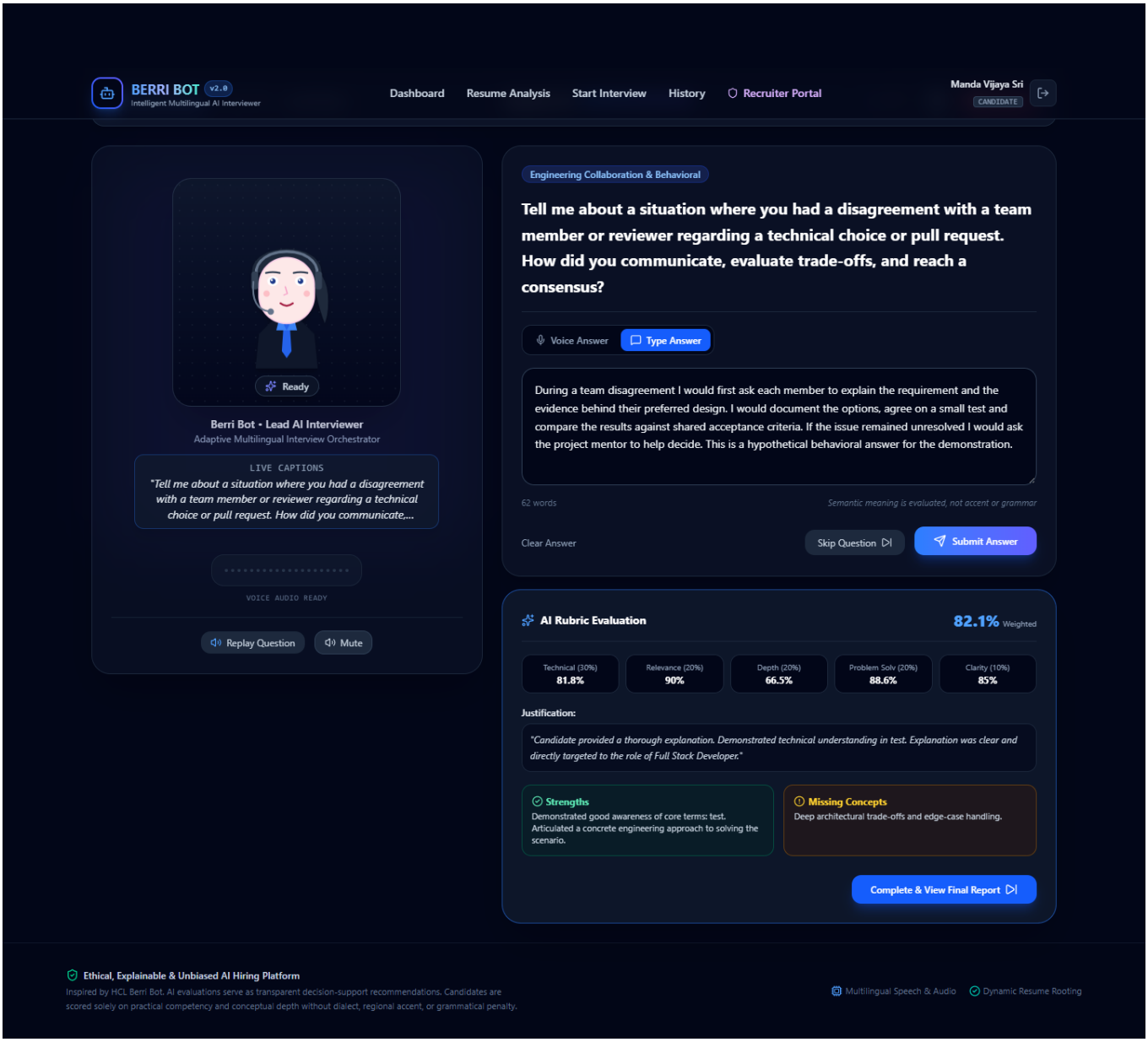


Figure 14 Completed interview results

The primary session reports 88 percent and Strong Match.

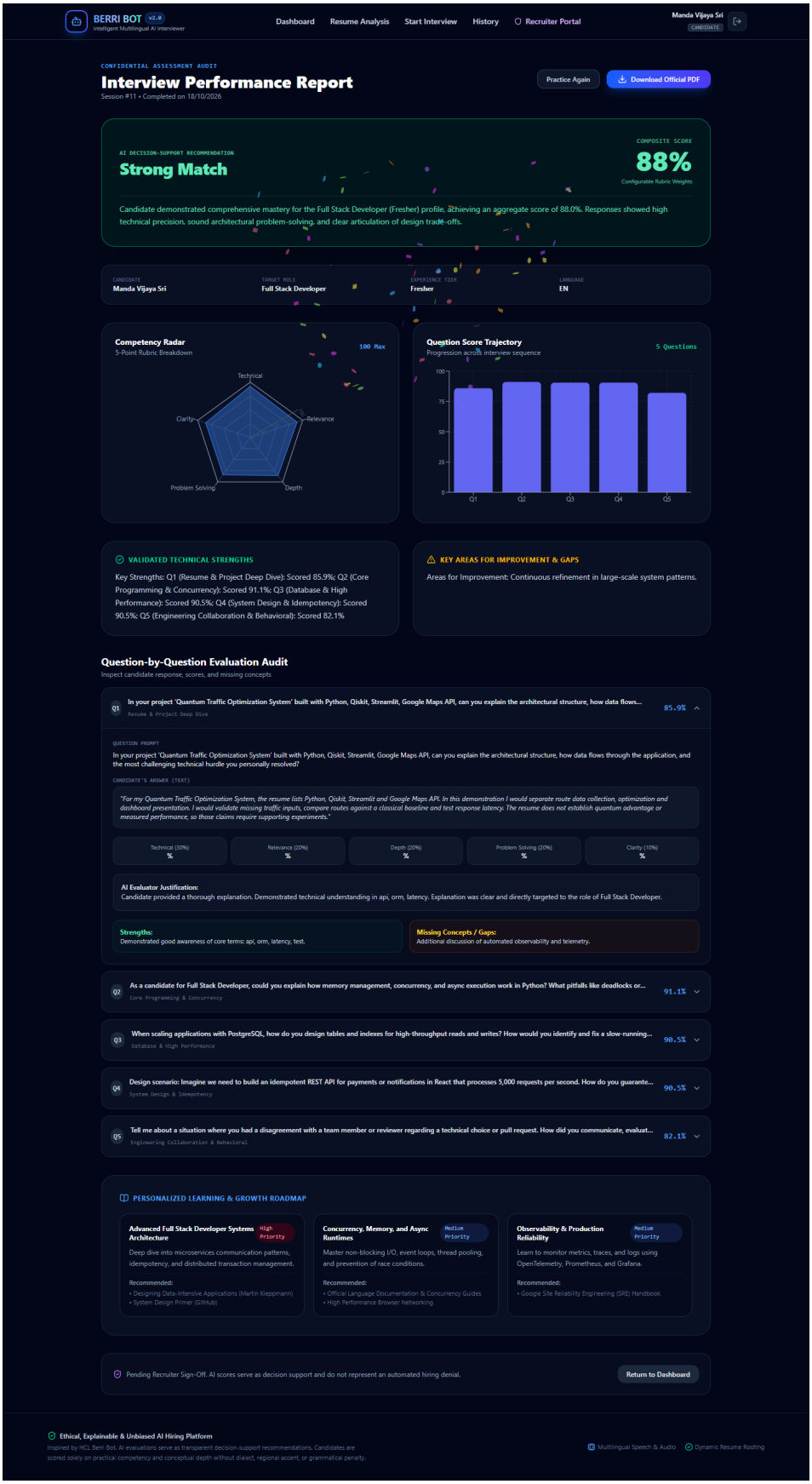


Figure 15 Repeated test history

The completed repeat appears alongside earlier exploratory test records.

